MKSSS's Cummins College of Engineering for Women, Pune
Department of Information Technology

MINI PROJECT REPORT

RepairWise — Intelligent Repair Decision System

A mobile app that helps people decide whether to repair, refurbish or replace a broken item, and tracks the money and pollution saved

Submitted by

UIT2024846	Akshita Pahade
UIT2024847	Shalaka Parhad
UIT2024848	Parul Deshmukh
UIT2024849	Samruddhi Pashankar

Under the guidance of
Dr. Dipti Patil

[Class / Division] [Academic Year 2026-27, please confirm]

Table of Contents

Abstract 1
1. Introduction and Problem Statement 1
1.1 Background 1
1.2 Problem statement 1
1.3 Objectives 1
1.4 Scope and target users 1
1.5 What is different about RepairWise 1
2. Sustainable Development Goals Addressed 1
3. Feasibility Analysis 1
3.1 Technical feasibility 1
3.2 Economic feasibility 1
3.3 Operational feasibility 1
3.4 Schedule feasibility 1
3.5 Legal, ethical and privacy considerations 1
3.6 Risks and mitigation 1
4. Requirements and Functionality 1
4.1 Functional requirements 1
4.2 Non-functional requirements 1
5. System Design 1
5.1 System architecture 1
5.2 Decision flow 1
5.3 Decision model 1
5.4 Safety gate 1
5.5 Second-life recommender 1
5.6 Data design 1
5.7 Screens and navigation 1
5.8 Key design decisions 1
6. Hardware and Software Configuration 1
6.1 Hardware 1
6.2 Software 1
6.3 Main libraries 1
6.4 Technology options considered 1
7. Implementation 1
7.1 Development approach 1
7.2 Project structure 1
7.3 Main modules 1
7.4 Development progress 1
7.5 Building the APK 1

7.6 Challenges and how they were solved 1
8. Demonstration of Working 1
8.1 Demonstration scenario 1
8.2 Screenshots of the developed app 1
8.3 Testing 1
9. Impact Analysis 1
9.1 Sustainability metrics 1
9.2 Method 1
9.3 Worked examples from the decision engine 1
9.4 Illustration of scale 1
9.5 Social and economic impact 1
9.6 Reliability of the impact figures 1
10. Limitations 1
11. Future Work 1
11.1 To be completed by the end of the semester 1
11.2 Longer-term scope 1
12. Conclusion 1
13. Contribution of Each Member 1
References 1
Appendix A: Category Data Used by the Decision Engine 1
Appendix B: Submission Details 1

If the page numbers above are missing, right-click the contents list in Word and choose Update Field.

Abstract

Every year, large numbers of phones, laptops, appliances and other electronic items are thrown away when they develop a fault, although many could be repaired at a fraction of the cost of a new one. People often cannot judge whether a repair is worth it, whether a technician's quote is fair, or whether the repair is safe to try at home. The result is avoidable e-waste, avoidable spending and avoidable carbon emissions.

RepairWise is a mobile application, built with React Native and Expo for Android, that gives a clear and explainable Repair, Refurbish or Replace verdict for a broken item. The verdict comes from a transparent weighted scoring model with five factors: repair cost against the price of a new item, remaining life, ease of repair, warranty, and environmental benefit. The app also includes a safety gate that stops users from attempting hazardous repairs, a second-life recommender (repair, sell, donate or recycle), DIY repair guides, a repair shop finder, an e-waste recycling guide, warranty and service reminders, PDF reports, dark mode, and English, Hindi and Marathi support. An impact dashboard shows the money saved, CO2 emissions avoided and e-waste kept out of the waste stream, and a repair outcome tracker records whether repairs last, which begins a dataset for future improvement of the model.

The app works fully offline, stores all data on the phone, and is delivered as an installable Android APK. It supports Sustainable Development Goals 12, 13, 8 and 11. This report covers the problem, feasibility, functionality, design, technology choices, implementation, demonstration, impact analysis, limitations, future work and each member's contribution.

1. Introduction and Problem Statement

1.1 Background

Electronic waste is one of the fastest-growing waste streams in the world (Ref. 2). Making a new device also carries a large hidden environmental cost, because much of a device's lifetime carbon footprint comes from manufacturing. Keeping a product in use for longer is therefore one of the most effective ways for an ordinary person to reduce waste and emissions. Governments and consumer groups are also promoting the right to repair (Ref. 4).

1.2 Problem statement

When an item breaks, people have no simple, trustworthy way to decide whether to repair or replace it. They do not know how a repair quote compares with the price of a new item, how much useful life the item has left, whether spare parts and technicians are easy to find, whether the repair is safe to attempt, or what the environmental difference would be. As a result, repairable items are discarded, people overspend, and e-waste and emissions increase.

1.3 Objectives

1. Give a clear Repair, Refurbish or Replace verdict for a broken item, with reasons the user can understand.
2. Make the decision explainable by showing the score for each factor and its weight.
3. Protect users by flagging repairs that should be left to a technician.
4. Suggest what to do with an item after the decision (keep, sell, donate or recycle).
5. Measure sustainability impact: money saved, CO2 avoided and e-waste kept out of the waste stream.
6. Record real repair outcomes so the app can improve with evidence in the future.
7. Work offline, on an ordinary Android phone, in English, Hindi and Marathi.

1.4 Scope and target users

The app is for everyday smartphone users in India who own common household items: phones, laptops, televisions, refrigerators, washing machines, air conditioners, mixer grinders, fans, headphones and bicycles, and any other item they add themselves. It supports a decision and does not replace a technician's diagnosis. The first version stores data on the device and does not need an account.

1.5 What is different about RepairWise

Repair guides, shop directories and carbon calculators exist as separate tools. RepairWise combines them at the moment an item breaks: one transparent decision that weighs cost and environmental benefit, a safety check, a second-life pathway, and a loop that records real outcomes. This combination is the contribution of the project. We do not claim that each individual part is new.

2. Sustainable Development Goals Addressed

RepairWise contributes to the following goals (Ref. 1). The mapping shows how the app's features connect to each goal.

SDG	How RepairWise contributes	App features
SDG 12 Responsible consumption and production (primary)	Prevents waste by extending the life of products. Target 12.5 aims to reduce waste generation through prevention, reduction, recycling and reuse.	Repair verdict, second-life options, recycling guide, DIY guides
SDG 13 Climate action	Repairing avoids the emissions of manufacturing and shipping a new item.	CO2 avoided estimate, carbon break-even months, impact dashboard
SDG 8 Decent work and economic growth	Directs repair work to local technicians and small repair businesses, and lets households keep money they would have spent on new items.	Repair shop finder, quote and cost comparison
SDG 11 Sustainable cities and communities	Reduces the e-waste that ends up in city waste systems (Target 11.6 covers municipal waste management).	E-waste recycling guide, data-wipe checklist

3. Feasibility Analysis

3.1 Technical feasibility

The app needs only on-device calculation, local storage and common phone features (camera, notifications, sharing). React Native with Expo supports all of these through maintained libraries, and Expo's cloud build service produces an Android APK without installing the Android SDK on the development laptop. The decision engine is a small, rule-based module and runs instantly on any phone. The team verified feasibility by building and running a working version on a real device through Expo Go and then as a standalone APK.

3.2 Economic feasibility

All tools used for the core app are free to use at student scale: VS Code, Node.js, Expo, GitHub and the Expo Go app. APK builds use Expo's free plan, which has build limits and queue times. The only optional cost is AI photo analysis, which needs a paid model API and a small server, so it is kept outside the core app. For users the app is free and needs no internet connection for its main functions.

3.3 Operational feasibility

The interface is a simple form followed by a clear verdict screen, with Hindi and Marathi available. No account or sign-up is needed. Since data stays on the phone, there is no server to maintain. The main operational risk is the quality of the built-in estimates, which the project addresses by labelling figures as estimates and by collecting real outcomes for future calibration.

3.4 Schedule feasibility

The project was developed in phases within the semester, starting with the core decision flow and then adding supporting features. Section 7.4 lists the milestones.

3.5 Legal, ethical and privacy considerations

- Personal data is stored only on the user's phone. Photos used for triage are not stored by the app.
- If the optional AI triage is enabled, the photo is sent to a server and an AI provider. Users must be told this and give consent before sending a photo.
- The app is decision support, not professional advice. Safety rules are fixed in code and cannot be overridden by an AI result.
- Repair shop listings in this version are sample data, clearly marked as such.

3.6 Risks and mitigation

Risk	Impact	Mitigation
Built-in cost, lifespan and carbon figures are approximate	Verdicts may be less accurate for some items	Label all figures as estimates; let users edit costs; record real outcomes for future calibration

Risk	Impact	Mitigation
User attempts an unsafe repair	Injury or damage	Safety gate with fixed rules; DIY options hidden for hazards and large appliances
AI service cost and key security	Unexpected cost or key misuse	AI is optional; key is held on a server, never inside the app; spending limit recommended
Free-plan limits for APK builds	Delays near the deadline	Test in Expo Go first and build the APK once features are stable
App behaves differently on different phones	Bugs on some devices	Test the installed APK on a separate phone; follow the test plan in Section 8

Conclusion of the feasibility study: the project is technically, economically and operationally feasible with free tools and a standard Android phone.

4. Requirements and Functionality

4.1 Functional requirements

ID	Requirement	Priority
FR1	The user can enter an item (from a list of categories, or a custom item) with age, new price, severity, repair quote, warranty and spare-parts availability.	Must
FR2	The app produces a Repair, Refurbish or Replace verdict with a score, confidence, reasons and factor breakdown.	Must
FR3	The app estimates money saved, CO2 avoided, e-waste avoided and carbon break-even months.	Must
FR4	The user can mark the outcome (repaired or replaced), which updates the impact totals.	Must
FR5	The app keeps a history of all checks with filters.	Must
FR6	A safety gate flags hazards and risky appliances and recommends a technician.	Must
FR7	The app suggests second-life options (keep, sell, donate, recycle).	Should
FR8	The user can record the result of a repair (still working or failed, cost, months), and the app schedules a six-month follow-up.	Should
FR9	An impact dashboard shows totals, monthly savings and repair-success statistics.	Must
FR10	DIY repair guides with safety warnings, a repair shop list, and an e-waste recycling guide.	Should
FR11	The user can set warranty or service reminders as local notifications.	Should
FR12	The user can share a PDF report of a decision and of the overall impact.	Should
FR13	Photo triage: the user adds a photo and picks or receives a damage assessment that pre-fills the check (AI analysis optional).	Should
FR14	Dark mode, and English, Hindi and Marathi language options.	Should
FR15	Sample data can be loaded for demonstration and cleared at any time.	Could

4.2 Non-functional requirements

- **Offline first:** all core features work without internet.
- **Explainability:** every verdict shows its reasons and factor scores.
- **Usability:** a short form, large touch targets, readable in light and dark themes.
- **Performance:** the verdict appears instantly, because the calculation runs on the phone.
- **Privacy:** data is stored only on the device; no account is needed.
- **Portability:** installable on Android phones as an APK.
- **Maintainability:** logic is separated from screens so the model can be changed without redesigning the interface.

5. System Design

5.1 System architecture

The app has four layers. The presentation layer holds the screens. The logic layer holds the decision engine, safety gate, second-life recommender, impact calculator, outcome statistics, language and theme modules. The data layer keeps decisions and settings in the phone's storage, with built-in datasets for categories, DIY guides and sample shops. Device services provide the camera, notifications, PDF sharing and links to maps and the phone dialler. An optional triage server and AI vision model sit outside the app and are used only if the user chooses photo analysis.

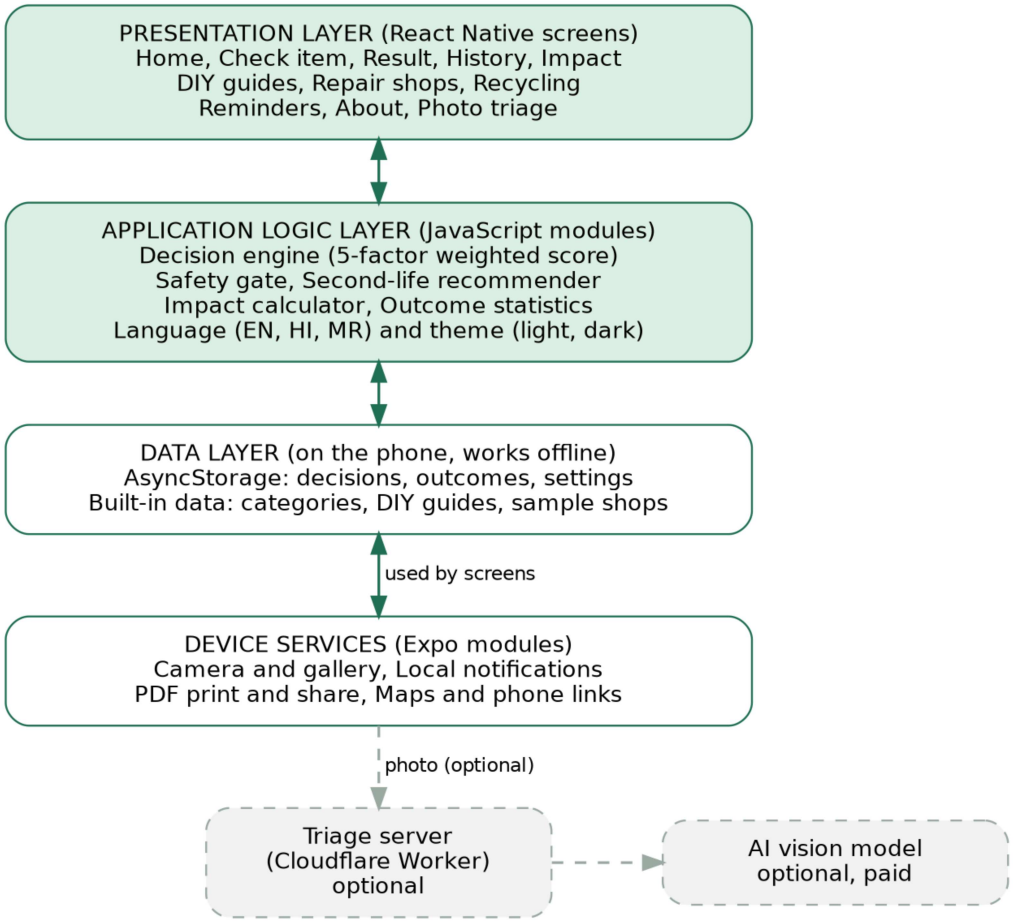


Figure 5.1: System architecture of RepairWise

5.2 Decision flow

Figure 5.2 shows how a check moves through the app, from the user's inputs to the saved result and the updated impact.

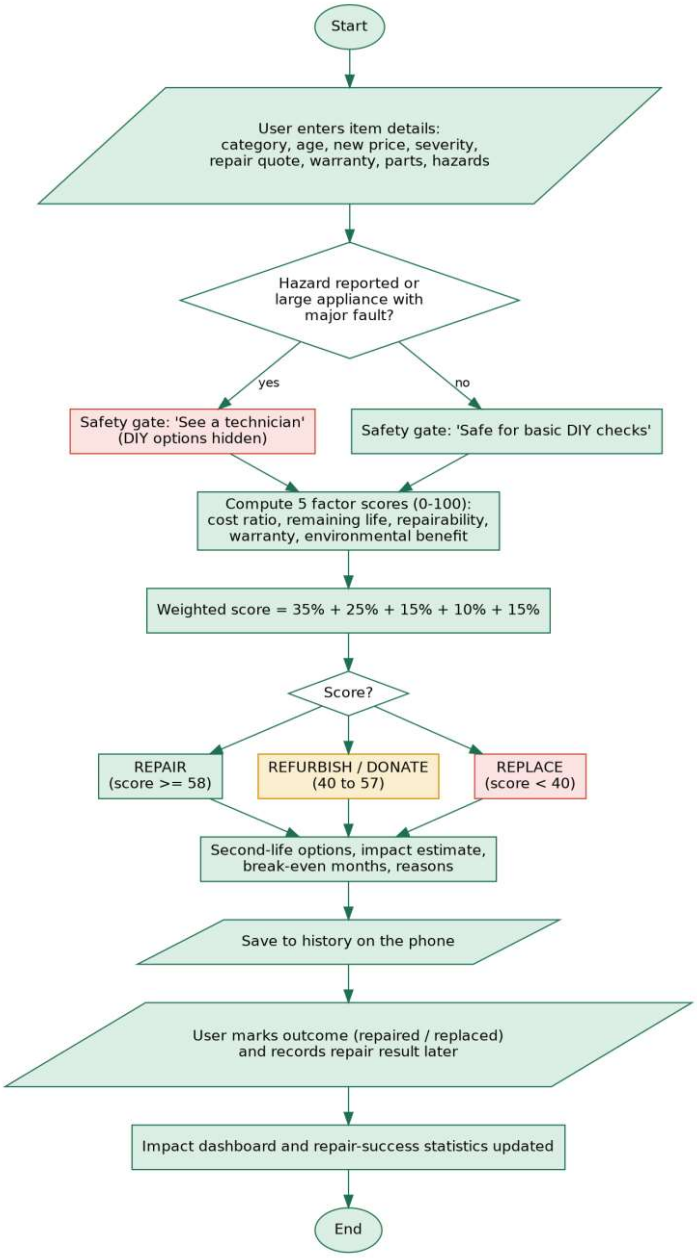


Figure 5.2: Flowchart of the repair decision process

5.3 Decision model

The verdict comes from a weighted score between 0 and 100, where a higher score favours repair. Each factor is first converted to a score from 0 to 100.

Factor	Weight	How its score is calculated
Repair cost vs new	35%	Ratio = repair cost ÷ price of a new item (repair cost is taken as 0 if under warranty). Score = 100 × (1 − (ratio − 0.2) ÷ 0.5), limited to 0–100. A ratio of 20% or less scores 100; 70% or more scores 0.
Remaining life	25%	Score = 100 × (1 − age ÷ typical lifespan of the category), limited to 0–100.

Factor	Weight	How its score is calculated
Ease of repair	15%	Base 80 if spare parts are easy to find, otherwise 40; then +10 for a minor fault, 0 for moderate, −20 for severe. Limited to 0–100.
Warranty	10%	100 if under warranty, otherwise 30.
Environmental benefit	15%	Score = 30 + (embodied carbon in kg CO2e ÷ 8), limited to 0–100, so items with higher manufacturing emissions favour repair more.

Score = 0.35 × cost + 0.25 × life + 0.15 × repairability + 0.10 × warranty + 0.15 × environment.

A score of 58 or more gives **Repair.** A score from 40 to 57 gives **Refurbish or donate**, meaning the repair is not a clear win and the user should consider a cheaper refurbishment, resale or donation. A score below 40 gives **Replace.** Confidence is shown as 55 + 0.8 × |score − 50|, with a ceiling of 95%, so it is higher when the score is far from the decision boundaries.

The carbon break-even is the number of months the repaired item must keep working for the repair to be worthwhile: repair cost ÷ (price of a new item ÷ typical lifespan in months). The weights and thresholds are design choices made by the team and are intended to be tuned using the outcome data collected by the app.

5.4 Safety gate

The safety gate is a fixed set of rules and is independent of the score:

- If the user reports any hazard (swollen battery, burning smell or sparks, liquid inside, exposed wires), the app says “Do not attempt this yourself. See a technician” and hides the DIY option.
- For televisions, air conditioners, refrigerators and washing machines with a moderate or severe fault, the same advice is shown, because of mains electricity, stored charge or gas.
- Otherwise the app shows “Safe for basic DIY checks”, with a reminder to stop if anything feels hot or smells burnt.

5.5 Second-life recommender

After the verdict, the app suggests what to do with the item. A Repair verdict suggests repairing and keeping it, and reselling or donating it later. A Refurbish verdict suggests selling to a refurbisher or donating, or giving it for parts. A Replace verdict suggests recycling at an authorised e-waste centre, and selling for parts if the item is still young. If a hazard was reported, the app suggests only a technician or an authorised recycler.

5.6 Data design

Data	Stored as	Contents
Decision record	List saved on the phone	Item details, inputs, result (score, verdict, factors, savings), safety result, hazards, outcome (pending, repaired, replaced), repair outcome log (worked, cost, months), date
Settings	Key-value on the phone	Language, dark mode
Reminders	List saved on the phone	Reminder name, scheduled date, notification id
Category table	Built into the app	Typical lifespan, new price, weight and embodied carbon per category (Appendix A)
DIY guides, sample shops	Built into the app	Eight guides with steps and warnings; eight sample shop listings

5.7 Screens and navigation

The app has five bottom tabs: Home, Check item, History, Impact and More. A result screen opens after a check. The More tab opens DIY guides, repair shops, reminders, e-waste recycling, About and SDGs, and also holds the dark mode switch and language choice. The Check item screen links to the photo triage screen. The interface uses a green and teal eco palette, rounded cards and a light and dark theme.

5.8 Key design decisions

- **Rule-based, not machine learning, for now.** A transparent model can be explained and checked. Machine learning needs real outcome data, which the app begins to collect.
- **Offline first.** No server is needed for the core app, which keeps it free to run and private.
- **Safety in code, not in AI.** Even if AI triage is used, hazards trigger fixed rules.
- **Estimates are labelled.** The app and report state that cost and carbon figures are approximate.

6. Hardware and Software Configuration

6.1 Hardware

Item	Purpose	Details
Development laptop	Writing code, running Expo, building and testing	Windows 11 laptop with limited free storage; [add RAM and processor]
Android phone (test)	Running the app in Expo Go and installing the APK	[Phone model and Android version]
Second Android phone	Testing the final APK on a clean install	[Phone model and Android version]
Internet connection	Expo cloud build, GitHub, installing packages	Not needed by the finished app for its core functions

6.2 Software

Software	Use
Windows 11, Command Prompt	Operating system and terminal
Visual Studio Code	Code editor
Node.js (LTS) and npm	JavaScript runtime and package manager
Expo SDK 57 and React Native	App framework and tooling (JavaScript)
Expo Go	Live testing on a phone during development
EAS Build (Expo cloud)	Building the Android APK
Git and GitHub	Version control and code sharing

6.3 Main libraries

Library	Use in the app
React Navigation (native stack, bottom tabs)	Screens and tab navigation
AsyncStorage	Saving decisions and settings on the phone
expo-notifications	Warranty, service and follow-up reminders
expo-print, expo-sharing	Creating and sharing PDF reports
expo-image-picker	Camera and gallery for photo triage
@expo/vector-icons	Icons
expo-font, safe-area-context, react-native-screens	Fonts, layout, native screens

6.4 Technology options considered

Option	Advantages	Why it was or was not chosen
Native Android (Java) with Android Studio	Team knows Java; full native features	Android Studio and the SDK need around 10–15 GB, and the team's laptops had limited free storage; slower setup
Web app packaged as an app (PWA / WebView)	Quick to build with HTML, CSS and JavaScript	Needs hosting and internet; weaker native features; less of a genuine mobile app
React Native with Expo (chosen)	Real mobile app, JavaScript, live testing on a phone, cloud APK build, no large local install	Chosen as the best fit for time, storage limits and features

7. Implementation

7.1 Development approach

The app was built in phases. The decision engine was written first as a standalone module with no user-interface code, so it can be tested and changed on its own. Screens were then added around it, and each phase was run on a real phone through Expo Go before the next phase began.

7.2 Project structure

File or folder	Purpose
App.js	Navigation, theme and language set-up
src/engine.js	Category data, decision engine, safety gate, second-life recommender, impact and outcome statistics
src/store.js	App state, saving and loading data, sample data
src/theme.js, src/i18n.js, src/ui.js	Colours and themes, translations, shared interface components
src/report.js, src/followup.js, src/config.js	PDF reports, follow-up notification, optional triage server address
src/data.js	DIY guides and sample shops
src/screens/	Home, Diagnose, Result, History, Impact, More, DIY, Shops, Recycle, Reminders, About, Triage
server/worker.js	Optional triage server code (not used by the core app)

7.3 Main modules

Decision engine

A pure function receives the item's details and returns the score, verdict, confidence, factor scores, plain-language reasons, money saved, CO_2 and e-waste avoided, and the break-even months. For a custom item, the user supplies the usual life and approximate weight, and the engine estimates embodied carbon as 20 kg CO_2e per kg of weight, with a minimum of 5 kg. This is a rough estimate that is clearly labelled.

Check item and Result screens

The Check item screen collects the inputs, estimates the repair cost from the severity if the user has no quote, and validates the entries. The Result screen shows the verdict card, safety banner, reasons, factor bars, second-life options, impact figures, next-step buttons, the outcome buttons and the repair outcome form.

Impact dashboard and outcome tracking

Totals count only items the user has marked as repaired. A monthly chart shows savings over six months. The repair outcome form stores whether the repair worked, its cost and how many months it has lasted, and the dashboard summarises the share of repairs still working and the

average months lasted. When an item is marked as repaired, the app schedules a local notification six months later asking how the repair went.

Photo triage

The triage screen lets the user take or choose a photo and pick what they see (for example a cracked screen, swollen battery or water damage). The choice sets the severity and any hazards on the check form. A server address can be configured to send the photo to an AI vision model for an automatic suggestion. In the submitted build the guided mode is the working path, and the AI path needs the optional server and a paid API.

Reports, reminders, language and theme

PDF reports are created from an HTML template and shared through the phone's share sheet. Reminders use local notifications. A translation layer maps interface text to Hindi and Marathi, and the theme module switches between light and dark colours.

7.4 Development progress

The table lists the milestones in order. Please add the dates from the GitHub commit history.

Phase	Work completed	Date
1	Decision engine, Home, Check item, Result, History, Impact dashboard, sample data, local storage	[date]
2	DIY guides, repair shop list, e-waste recycling guide, About and SDGs, More tab	[date]
3	Dark mode, PDF reports, warranty and service reminders	[date]
4	Hindi and Marathi language support	[date]
5	Custom "Other item" option	[date]
6	Safety gate, second-life options, repair outcome tracker and six-month follow-up, photo triage	[date]
7	APK build, installation on a second Android phone and testing	[date]

7.5 Building the APK

1. Set the app name, version and Android package name in app.json.
2. Check the project with the Expo doctor tool and fix package version mismatches.
3. Log in to Expo and run the EAS build for Android with a profile that produces an APK.
4. Download the APK from the Expo build page and install it on an Android phone.

7.6 Challenges and how they were solved

Challenge	Solution
Scripts blocked in Windows PowerShell, so the build tools would not run	Used Command Prompt instead, or changed the execution policy for the current user

Challenge	Solution
Android Studio needed more disk space than the laptops had free	Chose React Native with Expo and cloud builds
Some packages differed by a patch version from the Expo SDK	Ran the Expo install fix command and re-ran the doctor check
Notifications behave differently in Expo Go and in the installed app	Configured the notifications plugin and tested reminders in the APK
An AI key cannot be stored safely inside a mobile app	Designed the AI step to run through a separate server and kept it optional

8. Demonstration of Working

8.1 Demonstration scenario

1. Open the app and load the sample data from the Home screen.
2. Open Check item, choose Smartphone, enter an age, a price for a new phone and a repair quote, and tap Get my verdict.
3. Read the verdict, the reasons, the factor bars, the safety banner and the second-life options.
4. Tap “I repaired it” and fill in the Repair outcome form.
5. Open Impact to see money saved, CO2 avoided, e-waste avoided and the repair results.
6. Repeat with a swollen battery switched on to show the safety gate.
7. Show a DIY guide, the shop list, the recycling guide, dark mode, Hindi or Marathi, and a PDF report.

A short demonstration video follows this scenario. Video link: [add link].

8.2 Screenshots of the developed app

Replace each placeholder below with a screenshot from the installed app (Demo Mode is useful for filled screens).

[Paste screenshot here]	[Paste screenshot here]	[Paste screenshot here]	[Paste screenshot here]
Fig. 8.1: Home screen	*Fig. 8.2: Check item form*	*Fig. 8.3: Custom “Other item” fields*	*Fig. 8.4: Safety check switches*

[Paste screenshot here]	[Paste screenshot here]	[Paste screenshot here]	[Paste screenshot here]
Fig. 8.5: Result: Repair verdict	*Fig. 8.6: Result: reasons and score breakdown*	*Fig. 8.7: Result: safety banner and second-life options*	*Fig. 8.8: Result: repair outcome form*

[Paste screenshot here]	[Paste screenshot here]	[Paste screenshot here]	[Paste screenshot here]
Fig. 8.9: Photo triage screen	*Fig. 8.10: History with filters*	*Fig. 8.11: Impact dashboard*	*Fig. 8.12: Repair results recorded*

[Paste screenshot here]	[Paste screenshot here]	[Paste screenshot here]	[Paste screenshot here]
Fig. 8.13: More tab with language and dark mode	*Fig. 8.14: DIY guide expanded*	*Fig. 8.15: Repair shops (sample data)*	*Fig. 8.16: E-waste recycling guide*

[Paste screenshot here]	[Paste screenshot here]	[Paste screenshot here]	[Paste screenshot here]
Fig. 8.17: Reminders screen	*Fig. 8.18: Dark mode*	*Fig. 8.19: Hindi or Marathi interface*	*Fig. 8.20: PDF report*

8.3 Testing

The following manual test cases were prepared. Record the result for each on the installed APK.

ID	Test	Expected result	Status
T1	Load sample data	Home, History and Impact show sample items and totals	[Pass/Fail]
T2	Check a smartphone with a low repair quote	Repair verdict with score, reasons and factor bars	[Pass/Fail]

ID	Test	Expected result	Status
T3	Check an item under warranty	Warranty factor scores 100 and repair cost is treated as zero	[Pass/Fail]
T4	Choose Other item and leave the name empty	A message asks for the item name	[Pass/Fail]
T5	Complete an Other item check	Verdict is shown and the item appears in History	[Pass/Fail]
T6	Switch on Swollen battery	Red technician banner; DIY button hidden	[Pass/Fail]
T7	Check an air conditioner with major damage	Technician banner is shown	[Pass/Fail]
T8	Mark an item as repaired	Impact totals increase; outcome form appears	[Pass/Fail]
T9	Save a repair outcome	Impact shows repair results; Home check-in card disappears	[Pass/Fail]
T10	Switch dark mode on and off	Colours change across all screens	[Pass/Fail]
T11	Change language to Hindi and Marathi	Main screens change language	[Pass/Fail]
T12	Share a PDF report	Share sheet opens with a PDF	[Pass/Fail]
T13	Set a reminder for one minute later	Notification appears	[Pass/Fail]
T14	Photo triage with a guided option	Severity and hazards are filled in on the check form	[Pass/Fail]
T15	Turn on airplane mode and use the app	Core features still work	[Pass/Fail]
T16	Close and reopen the app	History and settings are kept	[Pass/Fail]

9. Impact Analysis

9.1 Sustainability metrics

Metric	Definition	SDG
Money saved (₹)	Price of a new item minus the repair cost, for items marked as repaired	12, 8
CO2 avoided (kg)	Estimated embodied carbon of manufacturing a new item, avoided by repairing	13, 12
E-waste avoided (kg)	Typical weight of the item that is kept out of the waste stream	12, 11
Carbon break-even (months)	How long a repaired item must keep working for the repair to be worthwhile	12, 13
Repair success rate (%)	Share of recorded repairs that were still working	12
Average months lasted	Average life of recorded repairs	12

9.2 Method

Impact is counted only when the user confirms that an item was repaired, so a verdict alone does not add to the totals. The figures come from the category table in Appendix A. These are approximate estimates for demonstration and must be replaced or checked against published data, such as the Global E-waste Monitor (Ref. 2), before being quoted as facts. Money saved is the price of a new item minus the repair cost, and it does not subtract any resale value of the old item.

9.3 Worked examples from the decision engine

The table shows the actual output of the app's decision engine for the six sample items.

Item	Age	New price	Repair quote	Score	Verdict	Saved if repaired	CO2 (kg)	E-waste (kg)
Redmi Note 11 (smartphone)	3 y	₹18,000	₹3,500	62	Repair	₹14,500	70	0.2
Dell Inspiron (laptop)	4 y	₹52,000	₹7,000	68	Repair	₹45,000	300	2.2
Mixer grinder	5 y	₹3,500	₹600	64	Repair	₹2,900	25	3
Sony 43 inch TV	9 y	₹32,000	₹19,000	25	Replace	₹13,000	400	10
boAt Rockerz (under warranty)	2 y	₹2,500	₹500	72	Repair	₹2,500	15	0.3
Ceiling fan	6 y	₹2,800	₹450	63	Repair	₹2,350	20	4

In the sample data, four items (phone, laptop, mixer grinder and headphones) were marked as repaired. Together they show **₹64,900 saved, 410 kg CO2e avoided and 5.7 kg of e-**

waste kept out of the waste stream. The television received a Replace verdict, and the fan is still pending. These sample figures illustrate how the dashboard works and are not measurements of real users.

9.4 Illustration of scale

Using the model's own assumptions, if 1,000 users each repaired one smartphone instead of replacing it, the model would count about 70 tonnes of CO2e and 200 kg of e-waste avoided. This is a hypothetical illustration of the model's arithmetic and is not a forecast or a measured result.

9.5 Social and economic impact

- Households avoid unnecessary spending, and the quote context helps them choose repairs with confidence.
- Local repair technicians gain customers who would otherwise buy new items.
- The safety gate lowers the chance of injury from unsafe home repairs.
- Awareness of reuse and responsible recycling grows through the guides and the recycling checklist.

9.6 Reliability of the impact figures

The weights, lifespans, prices and carbon values are estimates chosen by the team. Real values vary by brand, model and usage. The outcome tracker is designed to address this: as real repair costs and durations are recorded, they can be used to calibrate the model, which is described in Section 11.

10. Limitations

- Cost, lifespan and carbon figures are approximate and have not been validated against a large dataset.
- The decision model is rule-based and does not learn from data yet.
- Repair shop listings are sample data. The app opens maps to find real shops, but does not hold verified shop information.
- Photo analysis with AI needs a paid API and a server and is not enabled in the submitted build. The guided triage works without them.
- Data is stored on one phone only, so there is no backup or sync between devices.
- Some text, such as long guide steps and explanation sentences, is only in English.
- A photo shows only the outside of an item and cannot reveal internal faults, so triage is a guide and not a diagnosis.
- Notifications may behave differently on some phones depending on battery and permission settings.

11. Future Work

11.1 To be completed by the end of the semester

- Test the installed APK on more phones and fix any problems found.
- Complete Hindi and Marathi translations for the remaining screens.
- Check and cite sources for the lifespan, price and carbon values in the category table.
- Replace the sample shop data with verified local shops, or connect a places service.
- Review the decision weights using feedback from test users.

11.2 Longer-term scope

- **Learning from outcomes:** use anonymised repair results (model, issue, real cost, how long the repair lasted) to tune the weights and cost ranges, with consent and privacy protection.
- **Photo damage triage with AI:** connect the optional server so the app can suggest severity from a photo, always with human verification and fixed safety rules.
- **Verified technician network:** profiles, ratings, price transparency and booking for local repairers.
- **Take-back and recycling partnerships:** pickup scheduling with authorised recyclers and producer take-back programmes.
- **Institution dashboard:** colleges, housing societies and companies track e-waste avoided across their devices.
- **Repairability index:** a score per brand or model based on spare parts and documentation.
- **Voice input and more regional languages.**
- **Cloud backup and account sync** for users who want it.

12. Conclusion

RepairWise shows that a simple, transparent and offline mobile app can help people make better repair-or-replace decisions and can make the sustainability benefit of repairing visible. The project followed the full cycle of problem identification, feasibility analysis, functional design, system design, technology selection, implementation, demonstration and impact analysis. The result is a working Android app with an explainable decision engine, a safety gate, second-life guidance, an impact dashboard and a repair outcome tracker that starts to collect real evidence.

The app supports SDGs 12, 13, 8 and 11. Its figures are estimates, and the report states this openly. The next step is to validate the model with real outcome data and verified local services, so that the benefits can be measured and not only estimated. Through this project the team gained practical experience in mobile app development, decision modelling, user-centred design and sustainable computing.

13. Contribution of Each Member

Please complete this table so that it reflects the real work done by each member.

Roll No.	Name	Contribution
UIT2024846	Akshita Pahade	[Describe contribution, e.g. modules, screens, testing, report sections]
UIT2024847	Shalaka Parhad	[Describe contribution]
UIT2024848	Parul Deshmukh	[Describe contribution]
UIT2024849	Samruddhi Pashankar	[Describe contribution]

References

Please verify each reference and add access dates before submission.

1. United Nations, “Transforming our world: the 2030 Agenda for Sustainable Development,” General Assembly resolution, 2015.

2. International Telecommunication Union and UNITAR, “The Global E-waste Monitor 2024.”

3. Ministry of Environment, Forest and Climate Change, Government of India, “E-Waste (Management) Rules, 2022.”

4. Department of Consumer Affairs, Government of India, Right to Repair portal.

5. Expo documentation, https://docs.expo.dev

6. React Native documentation, https://reactnative.dev

7. React Navigation documentation, https://reactnavigation.org

Appendix A: Category Data Used by the Decision Engine

These values are approximate estimates used for demonstration. They are not measured data and should be checked against published sources.

Category	Typical life (years)	Typical new price	Weight (kg)	Embodied carbon (kg CO2e)
Smartphone	4	₹20,000	0.2	70
Laptop	6	₹55,000	2.2	300
TV	8	₹35,000	10	400
Refrigerator	12	₹30,000	60	500
Washing machine	10	₹28,000	55	400
Air conditioner	10	₹38,000	45	600
Mixer grinder	7	₹4,000	3	25
Fan	8	₹2,500	4	20
Headphones	3	₹3,000	0.3	15
Other item	5	₹5,000	2	40
Bicycle	10	₹12,000	14	120

Appendix B: Submission Details

Item	Details
GitHub repository	[paste link]
Android APK	[file name and download link]
Demo video	[paste link]
Presentation	[file name]
Group	UIT2024846, UIT2024847, UIT2024848, UIT2024849